

Internal-Lab Exam

NAME – M.Divya

ROLL NO – 2311CS040109

SUB – AGENTIC AI

9. Reasoning Model Benchmarking

Compare outputs across different prompting strategies

Code:

```
import ollama
```

```
import time
```

```
question = """
```

```
A train travels 60 km in 1 hour.
```

```
If it continues at the same speed, how far will it travel in 3 hours?
```

```
"""
```

```
prompts = {
```

```
    "Zero-Shot": question,
```

```
    "Few-Shot": """
```

```
Example:
```

```
Question: A car travels 20 km in 1 hour. How far does it travel in 2 hours?
```

```
Answer: 40 km.
```

Now solve:

```
""" + question,
```

```
    "Structured": """
```

Solve the problem using these steps:

1. Identify the given information.
2. Perform the calculation.
3. Give the final answer.

Question:

```
""" + question
```

```
}
```

```
for strategy, prompt in prompts.items():
```

```
    print("\nRunning:", strategy)
```

```
    start = time.time()
```

```
    response = ollama.chat(
```

```
        model="llama3.2",
```

```
        messages=[
```

```
            {
```

```
                "role": "user",
```

```
                "content": prompt
```

```
            }
```

```
        ]
```

```
)
```

```
end = time.time()
```

```
print("\n" + "=" * 50)
```

```
print(strategy)
```

```
print("=" * 50)
```

```
print(response["message"]["content"])
```

```
print(f"\nResponse Time: {end - start:.2f} seconds")
```

output:

```
PS C:\Users\matam\OneDrive\Desktop\labexam> ollama run llama3.2
/// bye
PS C:\Users\matam\OneDrive\Desktop\labexam> python benchmark.py

Running: Zero-Shot

=====
Zero-Shot
=====
To find the distance traveled in 3 hours, we need to multiply the speed of the train (60 km/h) by the time (3 hours).

Distance = Speed x Time
Distance = 60 km/h x 3 h
Distance = 180 km

So, the train will travel 180 km in 3 hours.

Response Time: 7.87 seconds

Running: Few-Shot

=====
Few-Shot
=====
To solve this, we need to find the speed of the train and then multiply it by 3 hours.

Speed = Distance / Time
Distance = 60 km
Time = 1 hour
```

```
PS C:\Users\matam\OneDrive\Desktop\labexam> python benchmark.py
```

Now, we need to find the distance traveled in 3 hours:

Distance = Speed x Time

Distance = 60 km/h x 3 hours = 180 km

Answer: The train will travel 180 km in 3 hours.

Response Time: 12.24 seconds

Running: Structured

```
=====
Structured
=====
```

To solve this problem, we will follow the steps below:

1. Identify the given information:
 - The train travels 60 km in 1 hour.
 - We need to find out how far it will travel in 3 hours.
2. Perform the calculation:
 - Since the train travels 60 km in 1 hour, its speed is 60 km/h.
 - To find the distance traveled in 3 hours, we multiply the speed by the time:
Distance = Speed x Time
Distance = 60 km/h x 3 h
Distance = 180 km
3. Give the final answer:
The train will travel 180 km in 3 hours.

```
PS C:\Users\matam\OneDrive\Desktop\labexam> python benchmark.py
```

```
Distance = Speed x Time
```

```
Distance = 60 km/h x 3 h
```

```
Distance = 180 km
```

3. Give the final answer:

The train will travel 180 km in 3 hours.

Response Time: 16.97 seconds

```
PS C:\Users\matam\OneDrive\Desktop\labexam> █
```

4. SQL Agent with Tool Use

Develop a ReAct-based agent using database tools

Code:

```
import ollama

import sqlite3

# -----
# Create SQLite database
# -----

conn = sqlite3.connect("database.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    marks INTEGER
)
""")

# Insert sample data

cursor.execute("DELETE FROM students")

students = [
```

```
(1, "Rahul", "CSE", 85),  
(2, "Priya", "ECE", 92),  
(3, "Arjun", "CSE", 78),  
(4, "Sneha", "IT", 88),  
(5, "Kiran", "CSE", 95)  
]
```

```
cursor.executemany(  
    "INSERT INTO students VALUES (?, ?, ?, ?)",  
    students  
)
```

```
conn.commit()
```

```
# -----  
# SQL Tool  
# -----
```

```
def execute_sql(sql):  
    try:  
        cursor.execute(sql)  
        result = cursor.fetchall()  
        return result  
  
    except Exception as e:  
        return f"SQL Error: {e}"
```

```
# -----
```

```
# SQL Agent
```

```
# -----
```

```
def sql_agent(question):
```

```
    prompt = f"""
```

```
You are a SQL assistant.
```

```
Database table:
```

```
students(
```

```
    id INTEGER,
```

```
    name TEXT,
```

```
    department TEXT,
```

```
    marks INTEGER
```

```
)
```

```
Convert the user's question into ONLY a valid SQLite SQL query.
```

```
Do not explain.
```

```
Do not use markdown.
```

```
Return only SQL.
```

```
User question:
```

```
{question}
```

```
"""
```

```
    response = ollama.chat(
```

```

model="llama3.2",

messages=[

    {

        "role": "user",

        "content": prompt

    }

]

)

sql = response["message"]["content"].strip()


# Remove markdown if model adds it

sql = sql.replace("`sql", "")

sql = sql.replace("`", "")

sql = sql.strip()


print("\nGenerated SQL:")

print(sql)


# Tool use

result = execute_sql(sql)


return result


# -----

# Main program

# -----

```

```
print("=====")

print("    SQL AGENT WITH TOOL USE")

print("=====")


while True:


    question = input("\nAsk a question (or type exit): ")


    if question.lower() == "exit":

        break


    result = sql_agent(question)


    print("\nTool Result:")

    print(result)


conn.close()
```

output:

```

PS C:\Users\matam\OneDrive\Desktop\labexam> cd question-04
PS C:\Users\matam\OneDrive\Desktop\labexam\question-04> python sql_agent.py
=====
SQL AGENT WITH TOOL USE
=====

Ask a question (or type exit): SQL AGENT WITH TOOL USE

Generated SQL:
SELECT * FROM students;

Tool Result:
[[1, 'Rahul', 'CSE', 85], (2, 'Priya', 'ECE', 92), (3, 'Arjun', 'CSE', 78), (4, 'Sneha', 'IT', 88), (5, 'Kiran', 'CSE', 95)]

Ask a question (or type exit): Show all students

Generated SQL:
SELECT * FROM students

Tool Result:
[[1, 'Rahul', 'CSE', 85], (2, 'Priya', 'ECE', 92), (3, 'Arjun', 'CSE', 78), (4, 'Sneha', 'IT', 88), (5, 'Kiran', 'CSE', 95)]

Ask a question (or type exit): Who has the highest marks?

Generated SQL:

```

```

PS C:\Users\matam\OneDrive\Desktop\labexam\question-04> python sql_agent.py

Generated SQL:
SELECT name FROM students ORDER BY marks DESC LIMIT 1

Tool Result:
[('Kiran',)]

Ask a question (or type exit): 

```

1. Text-to-SQL Workflow

Build an end-to-end LLM workflow with retrieval and query generation

Code:

```
import ollama
```

```
import sqlite3
```

```
# -----
```

```
# Create database
```

```
# -----
```

```
conn = sqlite3.connect("students.db")

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    department TEXT,
    marks INTEGER
)
""")

# Clear old data
cursor.execute("DELETE FROM students")

# Insert sample data
students = [
    (1, "Rahul", "CSE", 85),
    (2, "Priya", "ECE", 92),
    (3, "Arjun", "CSE", 78),
    (4, "Sneha", "IT", 88),
    (5, "Kiran", "CSE", 95)
]

cursor.executemany(
    "INSERT INTO students VALUES (?, ?, ?, ?)",
    students
```

)

conn.commit()

Text-to-SQL function

def text_to_sql(question):

prompt = f"""

You are a Text-to-SQL system.

Database table:

students(

id INTEGER,

name TEXT,

department TEXT,

marks INTEGER

)

Convert the following natural language question

into a valid SQLite SQL query.

Return ONLY the SQL query.

Do not explain.

Do not use markdown.

Question:

{question}

"""

```
response = ollama.chat(  
    model="llama3.2",  
    messages=[  
        {  
            "role": "user",  
            "content": prompt  
        }  
    ]  
)
```

```
sql = response["message"]["content"].strip()
```

```
# Remove markdown if generated
```

```
sql = sql.replace("```sql", "")
```

```
sql = sql.replace("```", "")
```

```
sql = sql.strip()
```

```
return sql
```

```
# -----
```

```
# Main program
```

```
# -----
```

```
print("=" * 50)

print("    TEXT-TO-SQL WORKFLOW")

print("=" * 50)


while True:


    question = input("\nEnter your question (or type exit): ")


    if question.lower() == "exit":

        break


    # Generate SQL

    sql = text_to_sql(question)


    print("\nGenerated SQL:")

    print(sql)


    # Execute SQL

    try:

        cursor.execute(sql)

        result = cursor.fetchall()


        print("\nQuery Result:")

        for row in result:

            print(row)


    except Exception as e:

        print("\nSQL Error:", e)
```

```
conn.close()
```

output:

```
PS C:\Users\matam\OneDrive\Desktop\labexam> cd question-01
PS C:\Users\matam\OneDrive\Desktop\labexam\question-01> python text_to_sql.py
=====
TEXT-TO-SQL WORKFLOW
=====

Enter your question (or type exit): TEXT-TO-SQL WORKFLOW

Generated SQL:
SELECT * FROM students;

Query Result:
(1, 'Rahul', 'CSE', 85)
(2, 'Priya', 'ECE', 92)
(3, 'Arjun', 'CSE', 78)
(4, 'Sneha', 'IT', 88)
(5, 'Kiran', 'CSE', 95)

Enter your question (or type exit): Show all students

Generated SQL:
SELECT * FROM students;

Query Result:
(1, 'Rahul', 'CSE', 85)
(2, 'Priya', 'ECE', 92)
(3, 'Arjun', 'CSE', 78)
(4, 'Sneha', 'IT', 88)
(5, 'Kiran', 'CSE', 95)

Enter your question (or type exit): Show students with marks greater than 85
```